Task 2 – Web Application Security Basics

Alfido Tech Cybersecurity Internship

Submitted By

Name: Eeshan Garg
Candidate ID: BS/REG/120082
Role: Cybersecurity Intern
Organization: Alfido Tech
Task: Task 2 – Web Application Security Basics
Tools Used: DVWA, XAMPP, Burp Suite, Web Browser
Date: May 2026

1. Objective

The objective of this task was to perform basic web application security testing using a vulnerable web application environment. The practical focused on identifying and demonstrating Cross-Site Scripting (XSS) vulnerabilities and understanding how attackers can exploit insecure input handling in web applications.

This task helped in understanding:

- Web application vulnerabilities
- Input validation weaknesses
- Client-side attacks
- Secure coding concepts
- Web security testing methodologies

2. Theory

What is Cross-Site Scripting (XSS)?

Cross-Site Scripting (XSS) is a web security vulnerability where attackers inject malicious scripts into trusted websites or web applications. These scripts execute inside the victim's browser when the vulnerable page is loaded.

XSS vulnerabilities can allow attackers to:

- Steal session cookies
- Hijack user sessions
- Modify webpage content
- Redirect users to malicious websites
- Execute malicious JavaScript code

3. Types of XSS

3.1 Reflected XSS

- Occurs when user input is immediately reflected in the web response.
- The malicious payload is not stored on the server.
- Usually delivered through crafted URLs.

3.2 Stored XSS

- The malicious payload is permanently stored on the server.
- Executes whenever another user visits the affected page.
- Considered more dangerous because it impacts multiple users.

3.3 DOM-Based XSS

- The vulnerability exists in client-side JavaScript.
- The attack happens within the browser DOM environment.
- The malicious payload may never reach the server.

4. Environment Setup

Software Used

- XAMPP Server
- DVWA (Damn Vulnerable Web Application)
- Web Browser

5. Implementation

Step 1 – Installing XAMPP

- Downloaded and installed XAMPP server.
- Started Apache and MySQL services.

Step 2 – Setting Up DVWA

- Downloaded DVWA.
- Extracted and renamed the folder as:

  `dvwa`
- Moved the folder into:

  `htdocs`
- Configured database credentials.

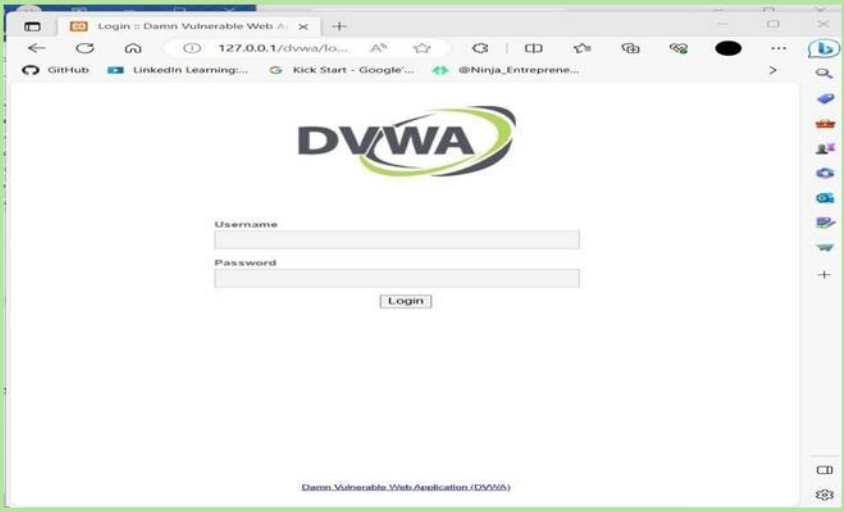
Credentials Used

```
Username: admin
Password: password
```

Step 3 – Accessing DVWA

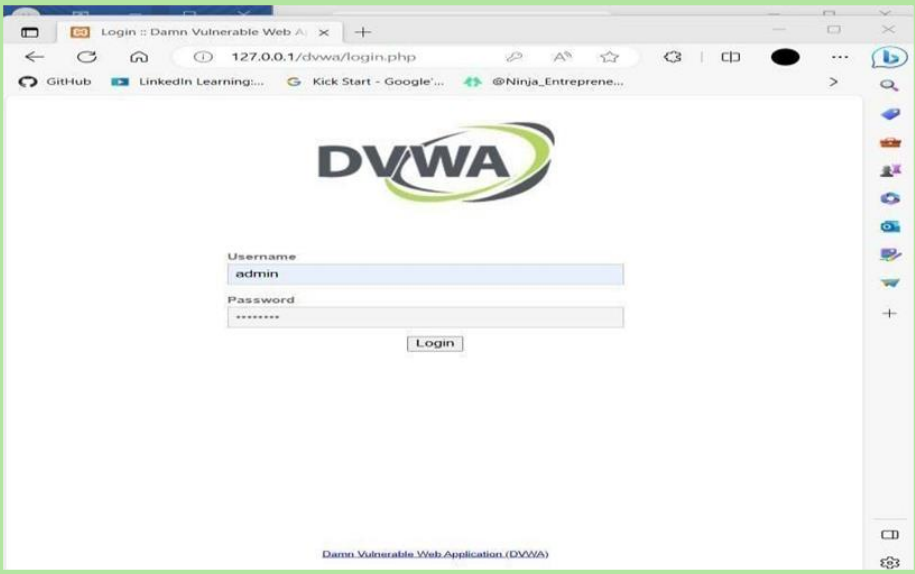
Opened the browser and accessed:

http://127.0.0.1/dvwa



Step 4 – Database Setup

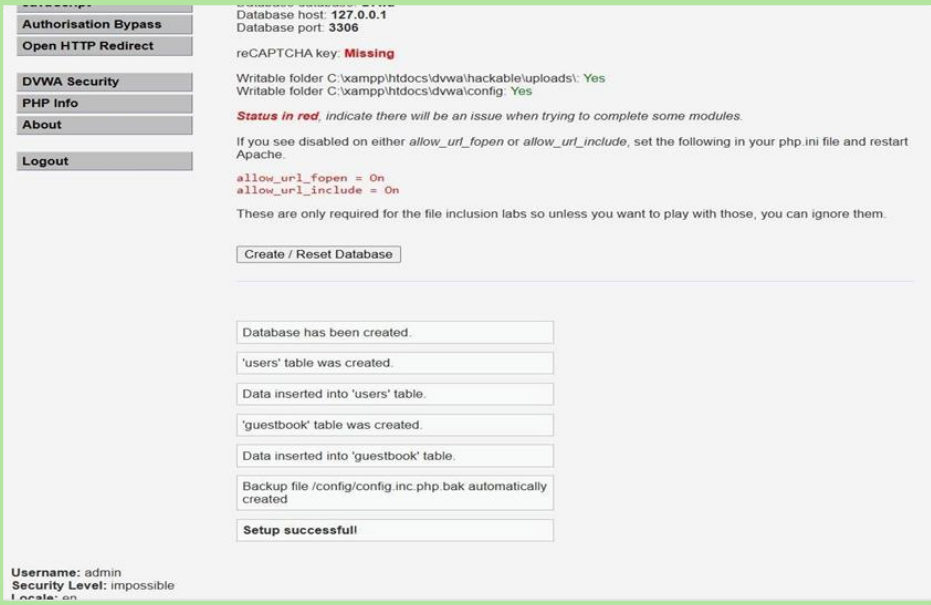
- Configured and initialized the DVWA database.
- Logged into the application using the provided credentials.



Step 5 – Configuring Security Level

- Reset the DVWA database.
- Set the security level to:

Low



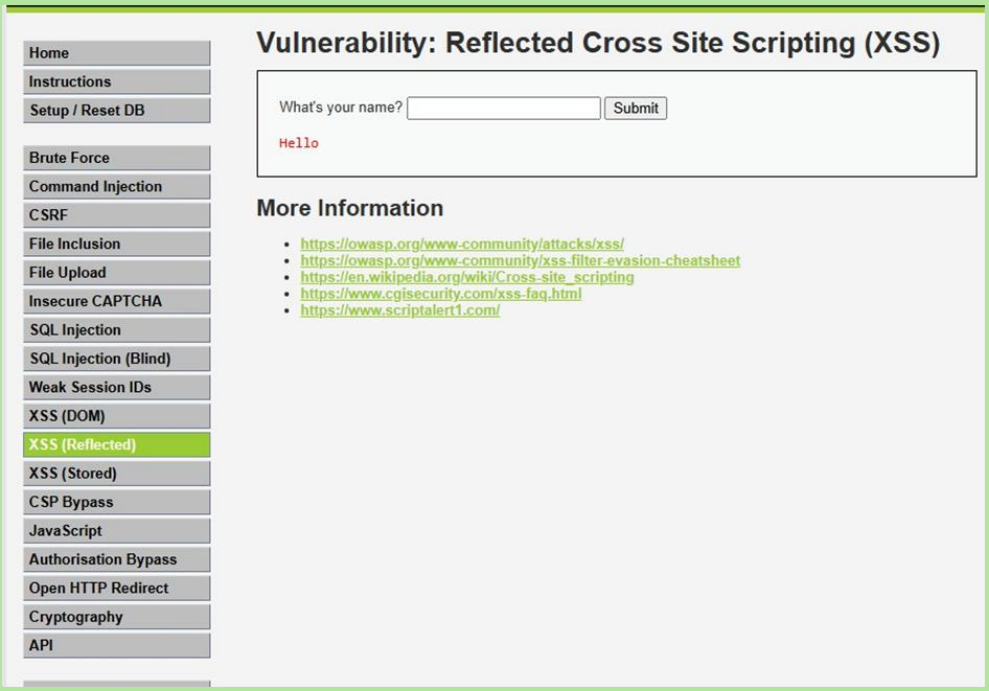
This allowed testing of vulnerable application behavior.

6. Reflected XSS Testing

Procedure

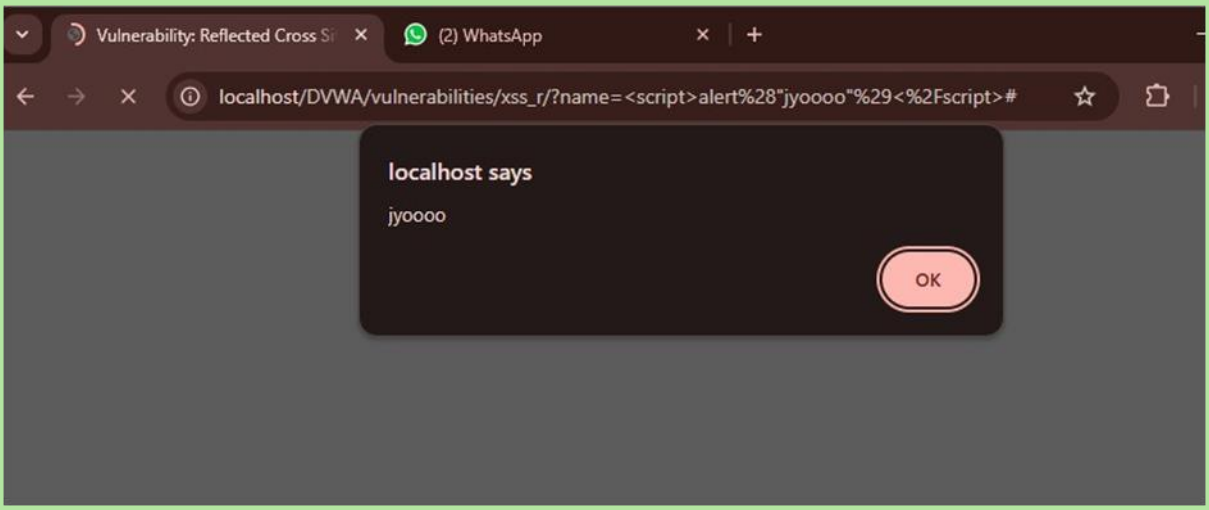
- Opened the “Reflected XSS” module.
- Entered a test name.
- Injected the following payload:

```
<script>alert("jyoooo")</script>
```



Observation

After refreshing the page, a JavaScript popup alert appeared, confirming successful execution of the injected script.



Result

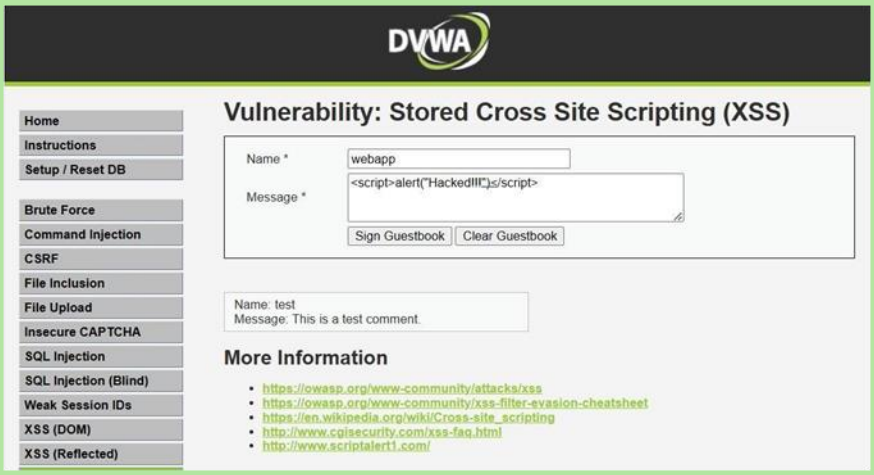
The application was vulnerable to Reflected XSS.

7. Stored XSS Testing

Procedure

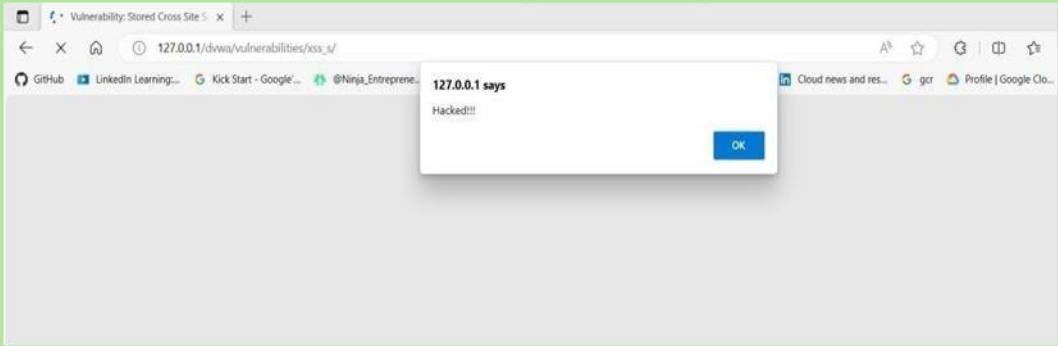
- Opened the “Stored XSS” module.
- Entered a name and message.
- Injected the following payload:

```
<script>alert("Hacked!!!")</script>
```



Observation

When the page was refreshed, the popup alert executed automatically.



Result

The application was vulnerable to Stored XSS.

8. DOM-Based XSS Testing

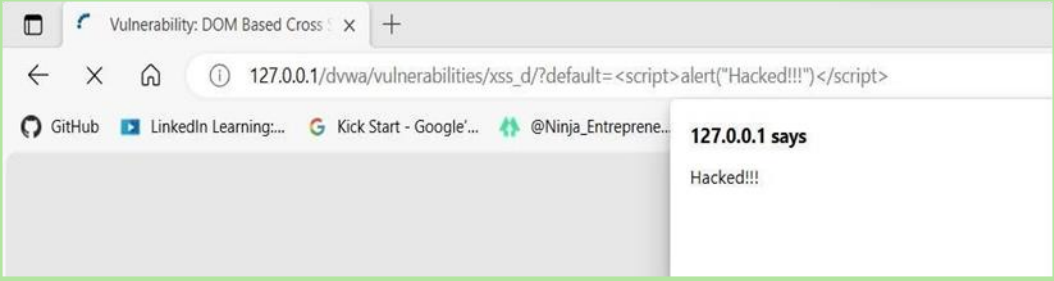
Procedure

- Opened the “XSS (DOM)” module.
- Modified the URL parameter using the following payload:

```
default=<script>alert("Hacked")</script>
```

Observation

The JavaScript alert executed successfully within the browser.



Result

The application was vulnerable to DOM-Based XSS.

9. Security Risks Identified

Vulnerability Type	Risk
Reflected XSS	Client-side script execution
Stored XSS	Persistent malicious code execution
DOM-Based XSS	Browser-side manipulation and exploitation

10. Mitigation Techniques

The following mitigation methods can help prevent XSS attacks:

1. Input validation and sanitization
2. Output encoding
3. Content Security Policy (CSP)
4. Secure session handling
5. Escaping special characters
6. Using modern web security frameworks

11. Conclusion

This practical demonstrated how Cross-Site Scripting vulnerabilities can be exploited within a web application environment. Different forms of XSS attacks, including Reflected, Stored, and DOM-Based XSS, were successfully identified and tested.

The exercise improved practical understanding of:

- Web application security
- Client-side attacks
- JavaScript injection
- Input validation weaknesses
- Secure coding practices

This task highlighted the importance of proper input sanitization and secure web development practices in preventing XSS vulnerabilities.